

Bangladesh University of Engineering and Technology

Department of Computer Science and Engineering

CSE 450 — Capstone Project

Group 15

Nafis Nahian	2105007
Abrar Zahin Raihan	2105009
Aritra Debnath	2105010
Tamzeed Mahfuz	2105012
Ruwad Naswan	2105051

Contents

1	Introduction	3
1.1	Project Background	3
1.2	Document Scope	3
2	Technical Specifications and Architectural Decisions	3
2.1	Architecture Overview	3
2.2	Technology Stack Decisions	4
2.3	Model Architecture Decisions	4
2.3.1	Tier 1: On-Device Model	4
2.3.2	Tier 2: Server-Side Ensemble	5
2.4	Security Architecture Decisions	5
3	System Requirements	6
3.1	Functional Requirements	6
3.1.1	User Registration and KYC	6
3.1.2	Liveness Detection	6
3.1.3	Capture Quality Assurance	7
3.1.4	Human Review Workflow	7
3.1.5	Continuous Learning	8
3.1.6	Administrative Functions	8
3.2	Non-Functional Requirements	9
3.2.1	Performance Requirements	9
3.2.2	Accuracy Requirements	9
3.2.3	Scalability Requirements	9
3.2.4	Availability and Reliability	10
3.2.5	Security Requirements	10
3.2.6	Compliance Requirements	10
3.2.7	Usability Requirements	11
3.2.8	Maintainability Requirements	11
4	Use Case Diagrams and Interactions	12
5	Software Modules and Input/Output Specifications	13
5.1	Module 1: Mobile Client SDK	13
5.2	Module 2: API Gateway Service	14
5.3	Module 3: Backend API Service	16
5.4	Module 4: ML Inference Service	16

5.5	Module 5: Database Access Layer	18
5.6	Module 6: Human Review Interface	19
6	Interaction Between Technology Layers	21
7	Data Flow Diagrams (DFD)	22
8	Database Schema Design	23
9	Conclusion	24

1 Introduction

1.1 Project Background

The rapid growth of digital financial services in emerging markets like Bangladesh has necessitated a paradigm shift from traditional knowledge-based authentication to robust biometric verification systems. However, the biometric landscape faces unprecedented challenges from generative AI technologies—GANs, diffusion models, and deepfakes—that can produce photorealistic synthetic identities capable of bypassing conventional liveness detection mechanisms.

Pathao Pay, as a leading mobile financial service provider in Bangladesh, requires a sophisticated fraud detection system that can:

1. Distinguish genuine live captures from AI-generated or manipulated images
2. Operate efficiently on resource-constrained mobile devices
3. Maintain low false positive rates to preserve user experience
4. Comply with ethical regulations and international standards
5. Adapt continuously to evolving generative AI techniques

1.2 Document Scope

This report presents a full system analysis and design specification for the proposed solution. It covers the technical architecture and technology stack selection (Section 2), functional and non-functional requirements (Section 3), use case modelling (Section 4), software module specifications with detailed input/output contracts (Section 5), inter-layer communication and end-to-end flow (Section 6), data flow diagrams (Section 7), database schema design (Section 8), and concluding remarks (Section 9).

2 Technical Specifications and Architectural Decisions

2.1 Architecture Overview

The system employs a **hybrid edge-cloud microservices architecture** with three primary tiers:

1. **Mobile Edge Layer:** On-device processing using optimized CNNs

2. **Cloud Processing Layer:** Server-side deep analysis with ensemble models
3. **Human Review Layer:** Manual verification for edge cases

2.2 Technology Stack Decisions

Component	Technology	Rationale
Mobile Frontend	React Native / Flutter	Cross-platform development, native performance, rich ecosystem
Backend API	FastAPI (Python)	High concurrency via async/await, automatic OpenAPI docs, ML integration
AI Framework	PyTorch + TorchScript	Production-grade model deployment, mobile optimization, ONNX export
Image Processing	OpenCV	Industry-standard computer vision library, hardware acceleration
Database	PostgreSQL	ACID compliance, JSON support, strong consistency guarantees
Cache Layer	Redis	Sub-millisecond latency, pub/sub for real-time updates
Message Queue	Celery + RabbitMQ	Async task processing, reliable delivery, horizontal scaling
API Gateway	NGINX / CloudFlare	Load balancing, rate limiting, DDoS protection
Monitoring	Prometheus + Grafana	Metrics collection, alerting, operational dashboards
Logging	ELK Stack	Centralized logging, full-text search, forensic analysis

Table 1: Technology Stack with Rationale

2.3 Model Architecture Decisions

2.3.1 Tier 1: On-Device Model

Selected Architecture: MobileNetV3-Small with custom classification head

Justification:

- Depthwise separable convolutions reduce parameters by 8-9x vs. standard CNNs

- Hardware-aware NAS optimization for ARM processors
- Squeeze-and-excitation blocks improve representational power
- Target: 3-5M parameters, 62MB RAM, 25ms inference on mid-range devices

Optimization Techniques:

- Post-training quantization (INT8) reducing model size by 4x
- TensorFlow-lite or ONNX runtime for deployment optimization

2.3.2 Tier 2: Server-Side Ensemble

Ensemble Components:

1. **Spatial Model:** EfficientNet-B2 fine-tuned on face anti-spoofing datasets
2. **Frequency Model:** ResNet-50 trained on FFT/DCT frequency representations
3. **Generative Detector:** Reconstruction-based autoencoder analyzing residual errors
4. **Attention Model:** Vision Transformer focusing on eyes, skin texture, boundaries

Fusion Strategy: Weighted averaging with learned confidence scores

2.4 Security Architecture Decisions

Security Aspect	Implementation
Authentication	JWT tokens with short expiry (15min), refresh token rotation
Data Encryption	AES-256 at rest for sensitive data
API Security	Rate limiting (100 req/min/IP), input validation, CORS policies
Biometric Privacy	On-device processing, raw image deletion after analysis, hash-based deduplication
Audit Logging	Immutable audit logs with cryptographic signatures

Table 2: Security Implementation Details

3 System Requirements

3.1 Functional Requirements

3.1.1 User Registration and KYC

- 1.1. A live selfie is captured directly from the user's mobile device camera.
- 1.2. Real-time guidance is delivered to the user to ensure optimal image quality with respect to lighting, distance, and angle.
- 1.3. On-device liveness screening is executed before upload to reduce bandwidth and early reject obvious spoofs.
- 1.4. If on-device confidence is insufficient, the captured image is transmitted to the server for deeper analytical processing.
- 1.5. The submitted live selfie is matched against the photograph on the user's National ID.
- 1.6. NID document authenticity is validated using format checks, security features, and issuer verification when available.
- 1.7. A final verification decision of PASS, FAIL, or REVIEW is issued, accompanied by a calibrated confidence score.
- 1.8. The user is notified of the outcome and provided a clear retry path when verification fails.

3.1.2 Liveness Detection

- 2.1. AI-generated faces produced by GAN architectures (StyleGAN, ProGAN, etc.) are identified and flagged.
- 2.2. Outputs from diffusion-based generative models (Stable Diffusion, DALL-E, Mid-journey) are recognized and rejected.
- 2.3. Deepfake face-swap manipulations are detected through learned visual inconsistency patterns.
- 2.4. Physical presentation attacks—including printed photographs, screen replays, and masks—are identified.
- 2.5. Frequency domain analysis is conducted to expose generation artifacts that are weak in the spatial domain.

- 2.6.** Camera sensor noise patterns are examined as an additional signal for authenticity assessment.
- 2.7.** Multiple independent detection signals are fused to arrive at a robust final verdict.
- 2.8.** High-risk sessions are re-checked with a stronger server-side model before a final decision.

3.1.3 Capture Quality Assurance

- 3.1.** Images falling below 480p resolution are automatically rejected prior to analysis.
- 3.2.** Blurry images are detected via a focus metric and rejected from further processing.
- 3.3.** Compliance is enforced such that the detected face occupies a minimum of 50% of the captured frame.
- 3.4.** Lighting conditions are assessed, and corrective feedback is surfaced to the user when quality is insufficient.
- 3.5.** Frames containing multiple faces are detected and handled with appropriate rejection or re-capture prompts.
- 3.6.** The system prevents submission if the capture appears to be a screen replay (screen glare, moiré).

3.1.4 Human Review Workflow

- 4.1.** Cases where model confidence falls below a configured threshold are automatically escalated to a prioritized human review queue.
- 4.2.** Reviewers are furnished with comprehensive analysis artifacts, including attention heatmaps, individual model scores, and session metadata.
- 4.3.** Case assignment is managed through a load-balanced distribution mechanism to ensure equitable reviewer workloads.
- 4.4.** Reviewer decisions and associated performance metrics are tracked and aggregated over time.
- 4.5.** A consensus mechanism is available for reviewers to collaboratively adjudicate high-stakes or ambiguous cases.
- 4.6.** Reviewer actions are auditable, and every manual decision is linked to a reviewer identity and timestamp.

3.1.5 Continuous Learning

- 5.1.** Challenging or ambiguous cases are flagged and routed into the dataset augmentation pipeline.
- 5.2.** Periodic model retraining is supported by incorporating newly collected and labeled attack samples.
- 5.3.** Model performance is continuously monitored to detect and respond to gradual degradation over time.
- 5.4.** New model versions are evaluated on offline benchmarks before deployment and can be rolled back safely.

3.1.6 Administrative Functions

- 6.1.** An administrative dashboard is provided for real-time monitoring of verification metrics and system health.
- 6.2.** Audit reports are generated on demand to support regulatory compliance documentation.
- 6.3.** Authorized administrators are able to tune detection thresholds without requiring system redeployment.
- 6.4.** Individual detection modules can be selectively enabled or disabled for operational flexibility.
- 6.5.** The full user account lifecycle including suspension and permanent deletion is managed through dedicated administrative controls.
- 6.6.** Administrative actions require multi-factor authentication and are logged for auditability.

3.2 Non-Functional Requirements

3.2.1 Performance Requirements

Metric	Target	Measurement Method
On-device inference	< 75ms (p95)	Profiling across device tiers
Server-side analysis	< 2s (p95)	End-to-end API latency
Total verification time	< 2.5s (p95)	User-perceived latency
Mobile memory footprint	< 62MB	Runtime memory profiling
Model size (on-device)	< 15MB	Deployment artifact size
API throughput	> 1000 req/s	Load testing with k6
Database query time	< 50ms (p99)	PostgreSQL query logs
Cache hit ratio	> 75%	Redis monitoring

Table 3: Performance Requirements

3.2.2 Accuracy Requirements

Metric	Target	Test Dataset
ACER (Average Classification Error Rate)	< 5%	Cross-dataset evaluation
EER (Equal Error Rate)	< 3%	ROC curve analysis
ROC-AUC	> 0.98	Holdout test set
True Positive Rate	> 97%	Real user samples
False Positive Rate	< 2%	Legitimate user testing
PAD (Presentation Attack Detection)	> 95%	ISO/IEC 30107-3 protocol
GAN Detection Rate	> 93%	StyleGAN3, ProGAN samples
Diffusion Detection Rate	> 90%	Stable Diffusion, Midjourney

Table 4: Accuracy Requirements

3.2.3 Scalability Requirements

- 3.1. Horizontal scaling must be supported to sustain stable operation under up to a 10x surge in traffic volume.
- 3.2. Performance targets are maintained even as the database grows to several million records.
- 3.3. Multi-region deployment is supported to achieve geographic distribution and minimize latency for end users.

- 3.4. Auto-scaling policies are implemented, triggering capacity adjustments in response to CPU and memory utilization thresholds.

3.2.4 Availability and Reliability

- 4.1. A minimum of 99.9% uptime.
- 4.2. Automated health checks are in place, with seamless failover activated upon detection of service degradation.
- 4.3. Full recovery from unexpected crashes is achieved within 60 seconds through automated restart and restoration.
- 4.4. Circuit breakers are deployed on all external dependency integrations to contain cascading failures.
- 4.5. Zero-downtime deployments are achieved via rolling updates.

3.2.5 Security Requirements

- 5.1. The system is aligned with ISO/IEC 30107-3 Presentation Attack Detection (PAD) guidance and test protocols.
- 5.2. Encryption is enforced for biometric data in transit and at rest.
- 5.3. Raw captured images are purged from storage within 24 hours of completing processing unless explicitly retained for review.
- 5.4. Role-based access control (RBAC) is enforced across all system interfaces and administrative functions.
- 5.5. All security-relevant events are persisted in an append-only, tamper-evident audit log.
- 5.6. Regular penetration testing exercises are conducted to identify and remediate security vulnerabilities proactively.
- 5.7. Keys and secrets are rotated on a defined schedule with emergency revocation procedures in place.

3.2.6 Compliance Requirements

- 6.1. Personal data is handled in accordance with applicable data protection law and GDPR principles where relevant.

- 6.2.** Comprehensive audit trails are retained for a minimum of 7 years to satisfy regulatory record-keeping obligations.
- 6.3.** Data portability is facilitated by enabling users to export their personal data upon request.
- 6.4.** Right-to-deletion requests are fulfilled within 30 days of submission, in compliance with applicable data protection law.
- 6.5.** Consent for biometric processing is explicitly collected and versioned per policy updates.

3.2.7 Usability Requirements

- 7.1.** Both Bengali and English are fully supported throughout the user interface.
- 7.2.** Error messages are clear and informative, each accompanied by actionable recovery guidance.
- 7.3.** A first-attempt verification success rate exceeding 85% is targeted across the user population.
- 7.4.** The complete verification flow is designed to conclude within no more than 5 user interactions.

3.2.8 Maintainability Requirements

- 8.1.** A minimum of 70% automated test coverage is enforced across all modules and critical execution paths.
- 8.2.** Comprehensive structured logging is implemented throughout, with correlation IDs enabling end-to-end request traceability.
- 8.3.** Feature flags are employed to support controlled, incremental rollout of new capabilities with minimal risk.
- 8.4.** Technical documentation is kept synchronized with the codebase and updated as part of every release process.
- 8.5.** Model and data versioning are maintained to ensure reproducibility and auditability.

4 Use Case Diagrams and Interactions

The following use case diagram captures the primary actors, system functions, and the relationships between them. It provides a high-level view of how end users, reviewers, and administrators interact with the verification system.

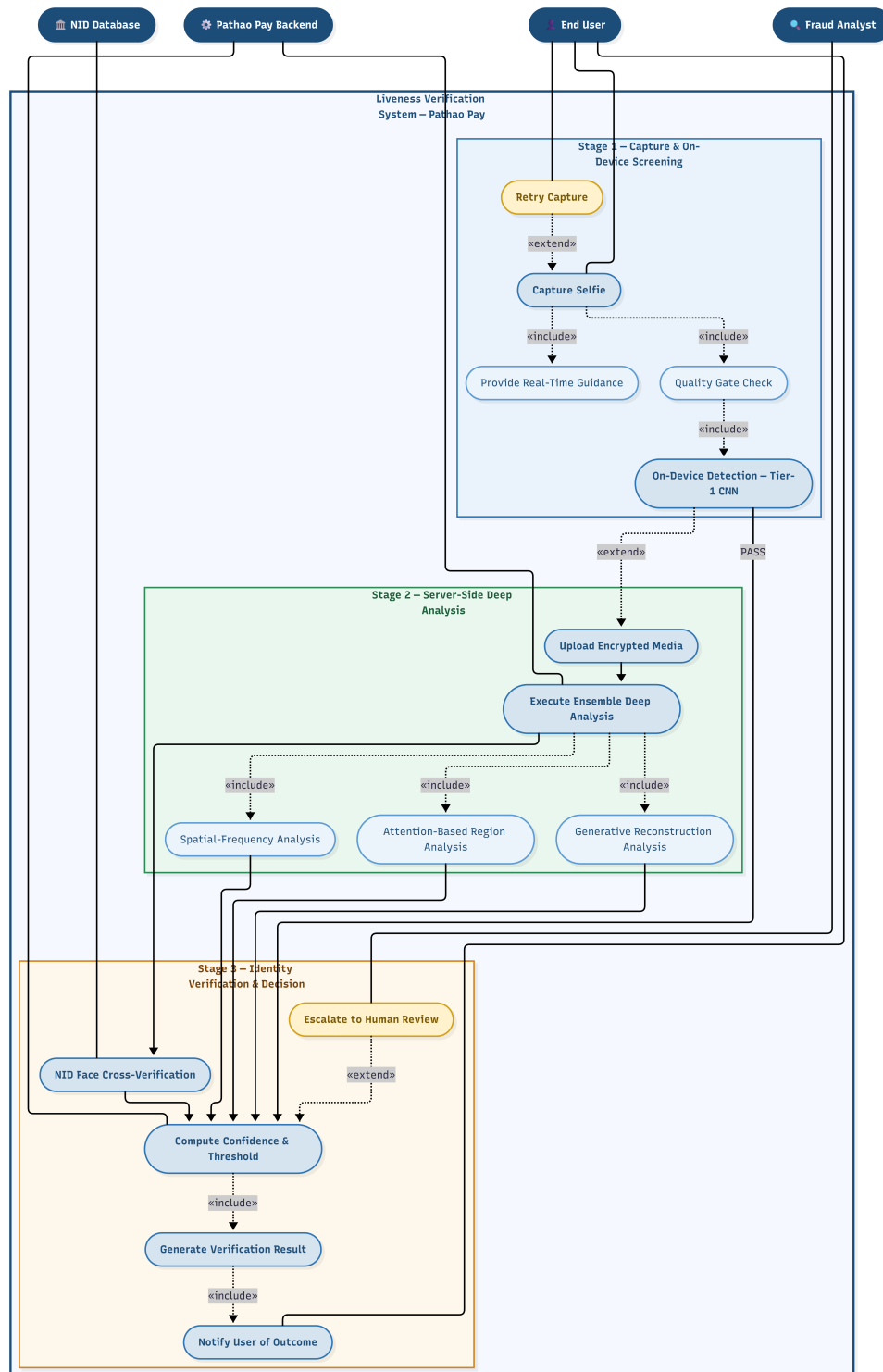


Figure 1: Use Case Diagram

5 Software Modules and Input/Output Specifications

The system is composed of six software modules, each with a well-defined boundary and responsibility. This section documents the purpose, internal components, and input/output specifications for each module.

5.1 Module 1: Mobile Client SDK

Purpose: Provides the user-facing interface and executes on-device liveness detection before any data is transmitted to the backend.

Components:

Mobile Client SDK — Internal Components	
Component	Responsibility
CameraController	Manages camera access, frame rate, and raw frame capture
GuidanceEngine	Delivers real-time on-screen feedback on lighting, distance, and angle
OnDeviceInference	Executes the MobileNetV3 model for fast on-device liveness scoring
APIClient	Handles authenticated communication with the backend
SessionManager	Manages the full lifecycle of a verification session

Input/Output Specifications:

Mobile Client SDK — Function I/O			
Function	Input	Type	Validation / Output
initSession()	userId, nidNumber	String, String	Required; format validated. Returns {sessionId, idempotencyKey, expiresAt}
captureFrame()	None	—	Returns {frame, timestamp, cameraInfo}
assessQuality()	frame: ImageData	uint8 [H, W, 3]	Resolution $\geq$ 480p. Returns {isValid, issues[], scores}
runInference()	preprocessed: Tensor	float32 [1, 224, 224, 3]	Normalised [0,1]. Returns {prediction, confidence, latency}
uploadMedia()	sessionId, imageBlob	String, Binary	Max 10 MB. Returns {mediaId, uploadUrl, status}

5.2 Module 2: API Gateway Service

Purpose: Acts as the single entry point for all client requests, enforcing authentication, rate limiting, and request routing before any traffic reaches the backend.

Components:

API Gateway Service — Internal Components	
Component	Responsibility
AuthMiddleware	Validates JWT tokens and authenticates incoming requests
RateLimiter	Throttles requests per IP and per user via Redis counters
RequestValidator	Sanitises input and enforces schema constraints
LoadBalancer	Distributes traffic across available backend instances
Logger	Records request and response data for audit and monitoring

Endpoint: POST /v1/liveness/sessions

Request Body:

```
1 {
2   "user_id":      "string (UUID)",
3   "nid_number":   "string (17 digits)",
4   "device_info": {
5     "os":         "Android | iOS",
6     "model":      "string",
7     "app_version": "string"
8   }
9 }
```

Response (201 Created):

```
1 {
2   "session_id":      "string (UUID)",
3   "idempotency_key": "string",
4   "expires_at":      "ISO8601 timestamp",
5   "status":          "CREATED"
6 }
```

Endpoint: POST /v1/liveness/sessions/{session_id}/stage1-result

Request Body:

```
1 {
2   "on_device_score": "float [0, 1]",
3   "inference_time_ms": "integer",
4   "quality_metrics": {
5     "resolution": "string (e.g. '1920x1080')",
6     "blur_score": "float",
7     "brightness": "float",
8     "face_size_ratio": "float"
9   }
10 }
```

Response (200 OK):

```
1 {
2   "status":      "STAGE1_SUBMITTED",
3   "should_upload": true,
4   "next_action": "UPLOAD_MEDIA | RETRY_CAPTURE"
5 }
```

5.3 Module 3: Backend API Service

Purpose: Implements the core business logic, managing session orchestration, media handling, task queuing, and result aggregation.

Components:

Backend API Service — Internal Components	
Component	Responsibility
SessionService	Manages the full verification session lifecycle
MediaService	Handles presigned URL generation and image storage coordination
QueueService	Enqueues asynchronous inference tasks via RabbitMQ/Celery
ResultAggregator	Combines stage-1 and stage-2 detection outputs into a final decision
NotificationService	Dispatches status notifications to users and reviewers

Key Function Signatures:

Backend API Service — Function I/O		
Function	Input Parameters	Return Type
create_session()	user_id: UUID nid_number: str device_info: DeviceInfo	Session object containing session_id, status, expires_at, idempotency_key
enqueue_stage2_processing()	session_id: UUID media_id: UUID priority: int = 0	TaskResult with task_id, status, enqueued_at, estimated_completion

5.4 Module 4: ML Inference Service

Purpose: Executes the ensemble of deep learning models responsible for liveness detection, produces a final confidence-weighted score, and generates interpretability artifacts for human review.

Components:

ML Inference Service — Internal Components	
Component	Responsibility
ModelRegistry	Manages model versioning, loading, and hot-swapping
Preprocessor	Resizes, normalises, and converts images to PyTorch tensors
InferenceEngine	Executes all four detection models in parallel
EnsembleAggregator	Applies weighted fusion to produce a single final score
FeatureExtractor	Generates Grad-CAM heatmaps and frequency visualisations

Function: `runEnsembleInference()`

The function accepts an `InferenceRequest` containing `session_id`, `media_id`, `image_url`, an optional `stage1_score`, and `device_metadata`. Processing proceeds through the following pipeline:

1. Image is downloaded from the provided S3 URL.
2. Preprocessing: resize to 224×224 , normalise to $[0, 1]$, convert to PyTorch tensor.
3. Four models execute in parallel:

Model	Architecture	Detection Focus
Spatial	EfficientNet-B2	Texture and pixel-level artefacts
Frequency	FFT + ResNet-50	Spectral generation signatures
Generative	Autoencoder	Reconstruction error for GAN/diffusion content
Attention	Vision Transformer (ViT)	Global semantic inconsistencies

4. Scores are fused via weighted averaging: $\text{final_score} = \sum_i w_i \cdot \text{score}_i$ where $\sum_i w_i = 1$
5. Grad-CAM heatmaps are generated for reviewer interpretability.

Decision Logic:

ML Decision Thresholds		
Condition	Decision	Outcome
<code>final_score</code> $\geq$ 0.8 and <code>confidence</code> $\geq$ 0.7	PASS	Identity verified; session completed
<code>final_score</code> $<$ 0.4 and <code>confidence</code> $\geq$ 0.7	FAIL	Presentation attack detected; session rejected
All other conditions	REVIEW	Escalated to human review queue

The function returns an `InferenceResult` containing: `final_score`, `confidence`, `individual_scores` per model, `decision`, `model_versions`, `inference_time_ms`, and `artifacts` (heatmaps and frequency visualisations).

5.5 Module 5: Database Access Layer

Purpose: Encapsulates all persistence logic to keep data access consistent, auditable, and isolated from business logic.

Core Components:

Component	Responsibility
SessionRepository	Creates, updates, and retrieves verification sessions
MediaRepository	Stores media metadata and storage references
ResultRepository	Persists model outputs and final decisions
ReviewRepository	Tracks human review cases and outcomes
AuditLogger	Writes append-only audit events for compliance

Table 5: Database Access Layer — Internal Components

SessionRepository — Method I/O:

Method	Input	Returns
<code>create(session)</code>	Session object	Persisted Session with assigned ID
<code>get_by_id(id)</code>	<code>session_id: UUID</code>	<code>Optional[Session]</code>
<code>update_status(id, status)</code>	UUID, SessionStatus	<code>bool</code> — success indicator
<code>get_expired_sessions()</code>	None	<code>List[Session]</code> past expiration
<code>delete_by_id(id)</code>	<code>session_id: UUID</code>	<code>bool</code> — soft delete confirmation

Table 6: SessionRepository — Method I/O

5.6 Module 6: Human Review Interface

Purpose: Provides a web-based dashboard through which reviewers assess flagged cases, view analysis artifacts, and issue a final manual decision.

Components:

Human Review Interface — Internal Components	
Component	Responsibility
CaseQueue	Displays flagged cases ordered by priority and submission time
ArtifactViewer	Renders the selfie, NID photo, heatmaps, and per-model scores
DecisionForm	Captures the reviewer's verdict, rationale, and confidence rating
Analytics	Tracks reviewer throughput, accuracy, and decision patterns over time

Input Presented to Reviewer:

Human Review Interface — Reviewer Input	
Artifact	Description
Selfie image	Original captured frame submitted by the user
NID photograph	Reference identity photo extracted from the National ID (if available)
On-device score	Stage-1 liveness score and confidence produced on the mobile device
Ensemble scores	Per-model scores from all four server-side detection models
Attention heatmaps	Grad-CAM visualisations highlighting suspicious facial regions
Frequency analysis	Spectral domain visualisation for GAN/diffusion artefact inspection
Device metadata	OS, device model, app version, and capture conditions
Historical attempts	Prior verification attempts associated with the same user account

Reviewer Output:

Human Review Interface — Decision Output		
Field	Type	Description
case_id	UUID	Unique identifier of the reviewed case
reviewer_id	UUID	Identity of the reviewer issuing the decision
decision	Enum	APPROVE, REJECT, or FLAG_FOR_INVESTIGATION
decision_rationale	String	Free-text justification for the decision
confidence_score	Integer	Reviewer self-assessed confidence on a scale of 1–5
tags	String[]	Labels such as poor_lighting or GAN_suspected
time_spent_seconds	Integer	Duration of case review in seconds
reviewed_at	ISO8601	Timestamp at which the decision was submitted

6 Interaction Between Technology Layers

To clearly illustrate how different architectural layers communicate during a high-risk transaction, the following sequence diagram represents the end-to-end interaction flow between the Presentation Layer (Mobile App), Application Layer (API Services), Machine Learning Layer (Liveness Face Matching Models), and Data Layer (Database Cache).

The diagram demonstrates request propagation, authentication, liveness verification, risk evaluation, and final decision return within the integrated Pathao payment ecosystem.

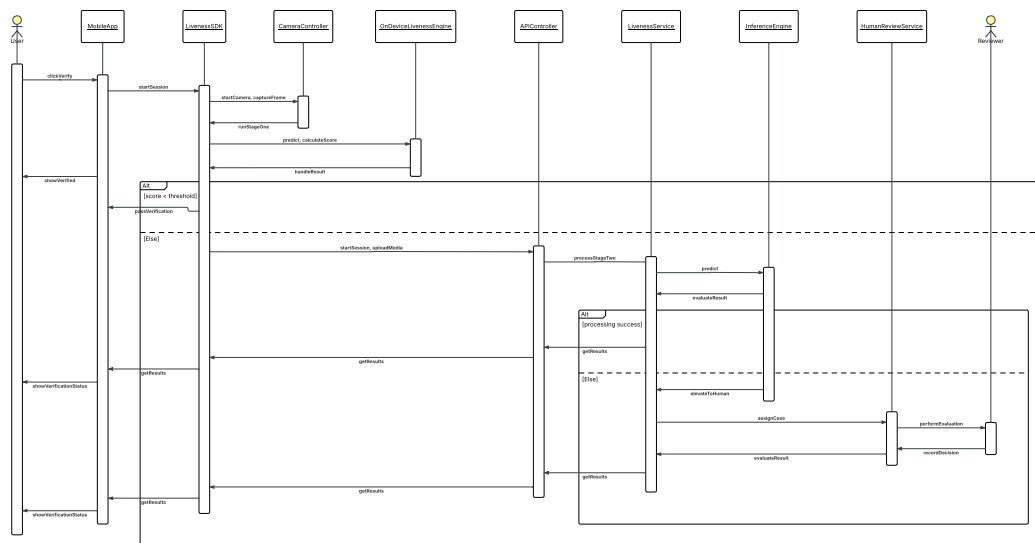


Figure 2: Sequence diagram

7 Data Flow Diagrams (DFD)

The Data Flow Diagram below illustrates how information propagates through the system, tracing the path from initial user input through processing stages to final output. It maps the interactions between processes, data stores, and external entities at each layer of the architecture.

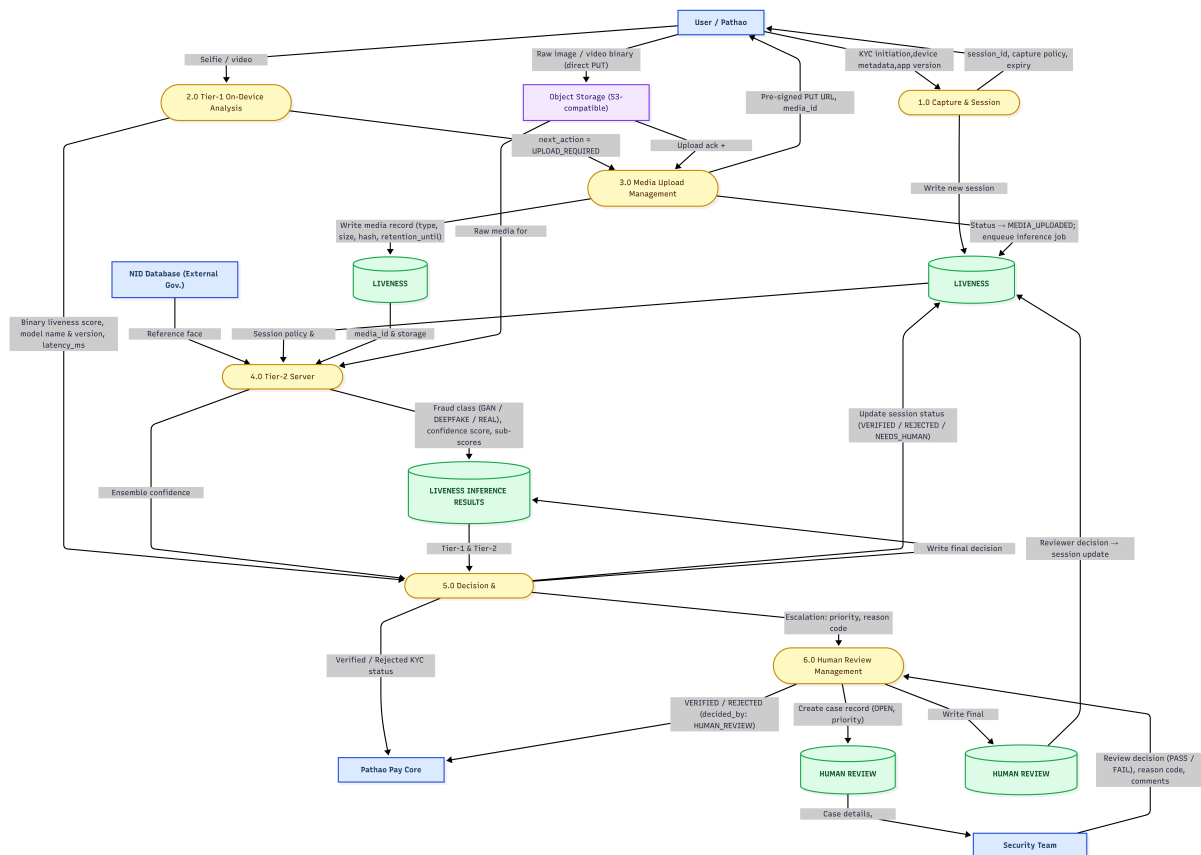


Figure 3: Data Flow Diagram

8 Database Schema Design

The Entity-Relationship Diagram below presents the proposed database schema, capturing the core entities, their attributes, and the relationships that govern data integrity across the verification pipeline.

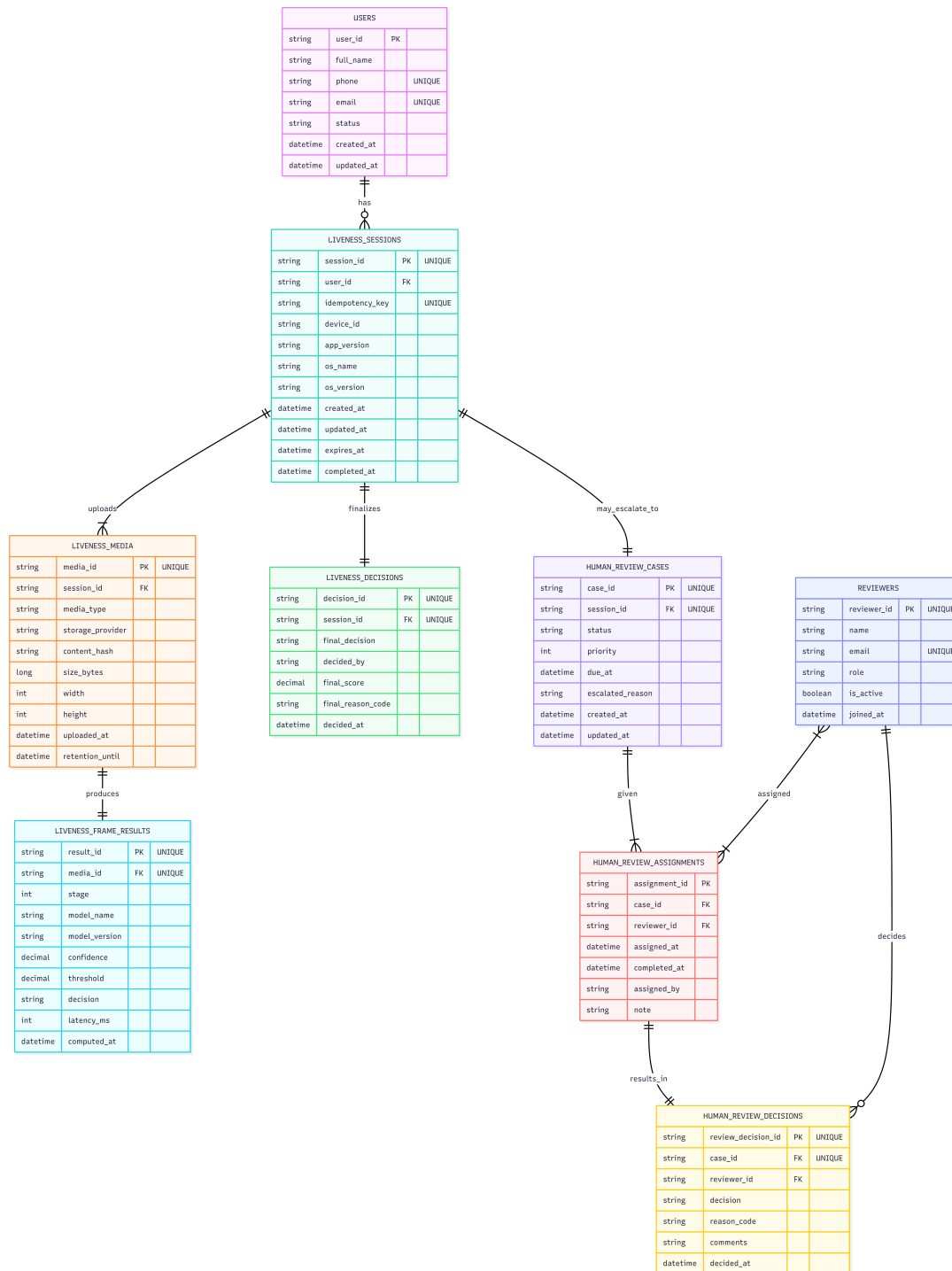


Figure 4: Entity-Relationship Diagram for Database Schema

9 Conclusion

In this system analysis and design report we have presented a comprehensive technical specification for an AI-Generated Image Detection system for secure liveness verification in Pathao Pay. The proposed architecture employs a three-tier detection framework balancing security, performance, and user experience through:

- Hybrid edge-cloud processing optimized for resource-constrained devices
- Ensemble learning combining multiple orthogonal detection paradigms
- Privacy-preserving architecture with on-device processing and minimal data retention
- Scalable microservices design supporting horizontal scaling
- Comprehensive monitoring and continuous improvement pipelines

The system is designed to meet stringent accuracy requirements ($ACER < 5\%$, $EER < 3\%$) while maintaining low latency ($< 2.5s$ end-to-end) and high availability (99.9% uptime). Compliance with ISO/IEC 30107-3 PAD standards and GDPR data protection regulations ensures both security and privacy.